PYTHON FOR BEGINNERS · SESSION 15

Exception Handling

try, except, finally, common errors and debugging tips.

Recap

Ask the class:

- What is the difference between mode `"w"` and mode `"a"`?
- Why do we use `with open(...)`?
- What error do you get when you open a file that is not there?

What is an exception?

An **exception** is an error that happens **while the program is running**. If nothing handles it, the program **stops** and prints an error message.

Try it: this crashes on purpose

```
print("Before")
print(10 / 0)
print("After")
```

ERROR

```
Before
  File "your code", line 2, in <module>
ZeroDivisionError: division by zero
```

- `Before` is printed, then the error stops the program.
- `After` **never runs**.
- The error type here is `ZeroDivisionError`.

Two kinds of errors

KIND	WHEN IT APPEARS	EXAMPLE
Syntax error	before the program runs: Python cannot read the code	`print("hi"`
Exception	while it runs: the code is fine but something goes wrong	`10 / 0`

Exception handling lets your program **deal with the problem** and keep going.

try and except

Put the risky code in `try`. Put the plan for a problem in `except`.

```
try:
    print(10 / 0)
except ZeroDivisionError:
    print("You cannot divide by zero")

print("The program keeps going")
```

OUTPUT

```
You cannot divide by zero
The program keeps going
```

- Python runs the `try` block.
- If an error happens, it **jumps** to the `except` block.
- If no error happens, the `except` block is **skipped**.
- Then the program continues after the whole `try` statement.

Code after the error is skipped

Work out the output first, then press Run to check

```
try:
    print("A")
    print(5 / 0)
    print("B")
except ZeroDivisionError:
    print("C")
print("D")
```

`B` is **skipped**. Once the error happens, the rest of the `try` block does not run.

Catching specific errors

Name the error you expect. Different errors can have different plans.

```
def divide(a, b):
    try:
        return a / b
    except ZeroDivisionError:
        print("b cannot be zero")
    except TypeError:
        print("both must be numbers")

print(divide(10, 2))
print(divide(10, 0))
print(divide(10, "x"))
```

OUTPUT

```
5.0
b cannot be zero
None
both must be numbers
None
```

- Python checks the `except` blocks **in order** and runs the **first match**.
- The failed calls print their message and then give back `None`.

The error message: as e

```
try:
    number = int("abc")
except ValueError as e:
    print("Problem:", e)
    print(type(e).__name__)
```

OUTPUT

```
Problem: invalid literal for int() with base 10: 'abc'
ValueError
```

`as e` gives a name to the **error object**. Printing it shows Python's own message.

One plan for several errors

```
values = ["12", "abc", None]

for v in values:
    try:
        print(int(v) * 2)
    except (ValueError, TypeError):
        print("cannot use", v)
```

OUTPUT

```
24
cannot use abc
cannot use None
```

Put the error names in a **tuple** to handle them the same way.

else and finally

BLOCK	WHEN IT RUNS
`try`	always first
`except`	only if an error happened
`else`	only if there was **no** error
`finally`	**always**, error or not

```
def show(x):
    try:
        result = 10 / x
    except ZeroDivisionError:
        print("error")
    else:
        print("result is", result)
    finally:
        print("done")

show(2)
print("---")
show(0)
```

OUTPUT

```
result is 5.0
done
---
error
done
```

- `else` keeps the "everything went well" code **separate** from the risky line.
- `finally` is for **clean-up**: closing a file, saying goodbye, releasing something.

finally runs even with return

Work out the output first, then press Run to check

```
def test():
    try:
        return "from try"
    finally:
        print("finally runs first")

print(test())
```

Common errors

ERROR	TYPICAL CAUSE	EXAMPLE
`SyntaxError`	broken Python grammar	`print("hi"`
`IndentationError`	wrong spaces at the start of a line	a body that is not indented
`NameError`	a name that does not exist	`print(total)` before `total = 0`
`TypeError`	wrong type for an operation	`"5" + 3`
`ValueError`	right type, wrong value	`int("abc")`
`ZeroDivisionError`	dividing by zero	`10 / 0`
`IndexError`	list position out of range	`[1, 2][5]`
`KeyError`	dictionary key not found	`{"a": 1}["b"]`
`AttributeError`	a thing has no such method	`"hi".push(1)`
`FileNotFoundError`	file is not there	`open("nope.txt")`
`ModuleNotFoundError`	module not installed or misspelt	`import mathh`

This box **causes** each of these on purpose, **catches** it, and prints the name of the error.

```python
tests = [
    lambda: print(total),
    lambda: "5" + 3,
    lambda: int("abc"),
    lambda: 10 / 0,
    lambda: [1, 2][5],
    lambda: {"a": 1}["b"],
    lambda: "hi".push(1),
    lambda: open("nope.txt"),
    lambda: __import__("mathh"),
]

for test in tests:
    try:
        test()
    except Exception as e:
        print(type(e).__name__, "-", e)
```

OUTPUT

```
NameError - name 'total' is not defined
TypeError - can only concatenate str (not "int") to str
ValueError - invalid literal for int() with base 10: 'abc'
ZeroDivisionError - division by zero
IndexError - list index out of range
KeyError - 'b'
AttributeError - 'str' object has no attribute 'push'
FileNotFoundError - [Errno 44] No such file or directory: 'nope.txt'
ModuleNotFoundError - No module named 'mathh'
```

SyntaxError cannot be caught

A `SyntaxError` happens **before** the program runs, so a `try` block cannot catch it in the same file. Fix the code instead.

Example: safe number input

Users type anything. A loop with `try` keeps asking until the input is valid.

safe_input.py (run it on your computer)

```
while True:
    text = input("Enter your age: ")
    try:
        age = int(text)
    except ValueError:
        print("Please type a whole number.")
        continue
    if age < 0:
        print("Age cannot be negative.")
        continue
    break

print("Your age is", age)
```

OUTPUT

```
Enter your age: abc
Please type a whole number.
Enter your age: -5
Age cannot be negative.
Enter your age: 20
Your age is 20
```

The same idea with a list of "typed" answers

```
answers = ["abc", "-5", "20"]

for text in answers:
    try:
        age = int(text)
    except ValueError:
        print(text, "is not a whole number")
        continue
    if age < 0:
        print(age, "is negative")
        continue
    print("Your age is", age)
    break
```

OUTPUT

```
abc is not a whole number
-5 is negative
Your age is 20
```

A function you can reuse

```
def to_number(text):
    try:
        return float(text)
    except ValueError:
        return None

print(to_number("3.5"))
print(to_number("hello"))
print(to_number(""))
```

OUTPUT

```
3.5
None
None
```

Raising your own errors

You can **raise** an exception yourself with `raise`, when your code finds something wrong.

```
def set_age(age):
    if age < 0:
        raise ValueError("age cannot be negative")
    return age

try:
    set_age(-3)
except ValueError as e:
    print("Caught:", e)

print(set_age(20))
```

OUTPUT

```
Caught: age cannot be negative
20
```

- Choose the **right type**: `ValueError` for a bad value, `TypeError` for a bad type.
- Write a **clear message** so the next person knows what went wrong.
- The function that finds the problem **raises**. The code that calls it decides what to do and **catches**.

Your own exception class

```
class LowBalanceError(Exception):
    pass

def withdraw(balance, amount):
    if amount > balance:
        raise LowBalanceError("not enough money")
    return balance - amount

try:
    print(withdraw(100, 30))
    print(withdraw(100, 500))
except LowBalanceError as e:
    print("Sorry:", e)
```

OUTPUT

```
70
Sorry: not enough money
```

A custom exception is a class that **inherits from** `Exception`. We learn classes properly later; for now just copy this pattern when you want an error with your own name.

Exceptions and files

Last class we checked whether a file exists. A `try` block is the better way: it handles the missing file **and** other problems.

```
def read_file(name):
    try:
        with open(name) as f:
            return f.read()
    except FileNotFoundError:
        return "(file not found)"

with open("hello.txt", "w") as f:
    f.write("Hello!")

print(read_file("hello.txt"))
print(read_file("missing.txt"))
```

OUTPUT

```
Hello!
(file not found)
```

Reading numbers from a file safely

```
with open("scores.txt", "w") as f:
    f.write("80\nabc\n95\n\n70\n")

total = 0
bad = 0
with open("scores.txt") as f:
    for line in f:
        try:
            total += int(line)
        except ValueError:
            bad += 1

print("Total:", total)
print("Bad lines:", bad)
```

OUTPUT

```
Total: 245
Bad lines: 2
```

- The bad line `abc` and the blank line are **skipped**, and the loop carries on.
- Without `try`, the first bad line would stop the whole program.

finally with a file opened by hand

```
f = open("temp.txt", "w")
try:
    f.write("saving...")
    print(10 / 0)
except ZeroDivisionError:
    print("something went wrong")
finally:
    f.close()
    print("file closed:", f.closed)
```

OUTPUT

```
something went wrong
file closed: True
```

`with` does this job for you, but `finally` is the idea behind it: **clean up no matter what**.

Catching too much

Do not do this

```
try:
    total = pric * 2
except:
    print("something went wrong")
```

A **bare** `except:` catches **every** error, even your own typing mistakes. Here `pric` is a typo for `price`, but the program only says "something went wrong", and you never learn why.

- Catch the **specific** error you expect.
- If you must catch everything, write `except Exception as e:` and **print or log** `e`.
- Keep the `try` block **small**: only the line that can fail.
- Never write `except: pass`. It hides bugs.

```
try:
    total = pric * 2
except Exception as e:
    print(type(e).__name__, "-", e)
```

OUTPUT

```
NameError - name 'pric' is not defined
```

Reading a traceback

When a program crashes, Python prints a **traceback**. Do not be scared of it. It tells you exactly what happened.

Try it: this crashes on purpose, then read the message

```
def average(numbers):
    return sum(numbers) / len(numbers)

def report(marks):
    avg = average(marks)
    print("Average:", avg)

report([])
```

ERROR

```
  File "your code", line 2, in average
ZeroDivisionError: division by zero
```

1. **Read the last line first.** It names the error and gives the reason: `ZeroDivisionError: division by zero`.

2. The lines above show **where**: each `File ... line N, in name` is one step of the call chain.
3. The **last** step before the error is where the crash happened (inside `average`).
4. The steps above it show **how we got there** (`report` called `average`).

The real cause is that `marks` was empty, so `len(numbers)` was `0`. The error shows in `average`, but the **fix** may belong in the code that called it.

Debugging tips

Debugging means finding and fixing the cause of a bug. A calm method beats guessing.

1. **Read the error.** Last line first, then the line number.
2. **Reproduce it.** Find the smallest input that makes it fail.
3. **Print the values.** Show variables just before the failing line.
4. **Check the types.** `print(type(x))` solves many `TypeError` problems.
5. **Split the problem.** Test one function at a time with easy inputs.
6. **Change one thing at a time**, then run again.
7. **Explain it aloud** to a friend or a rubber duck. You often spot it while talking.
8. **Take a break.** A fresh look finds what a tired one missed.

Print debugging

```
def total_price(prices, discount):
    total = 0
    for p in prices:
        total += p
    print("DEBUG total before discount:", total)
    total = total - total * discount
    print("DEBUG total after discount:", total)
    return total

print(total_price([100, 200, 50], 10))
```

```
OUTPUT
DEBUG total before discount: 350
DEBUG total after discount: -3150
-3150
```

The debug lines show the total is `350`, then `-3150`. The bug is clear: `discount` should be `0.10`, not `10`. Remove the debug prints when you are done.

Common bug patterns

- **Off by one:** `range(5)` stops at `4`.
- `=` **instead of** `==` in a condition.

- **Changing a list while looping over it.**
- **Forgetting** `return`, so the result is `None`.
- **Comparing text with a number:** `"5" == 5` is `False`.

assert

`assert` checks that something is **true**. If it is false, Python raises an `AssertionError`. It is a quick way to test your own code.

```
def square(n):
    return n * n

assert square(4) == 16
assert square(-3) == 9
print("All tests passed")

try:
    assert square(5) == 20, "square(5) should be 25"
except AssertionError as e:
    print("Test failed:", e)
```

```
OUTPUT
All tests passed
Test failed: square(5) should be 25
```

- `assert condition, "message"` shows the message if the condition is false.
- Use `assert` to check your **own** code. Use `try` and `except` for problems that can happen in real use, like bad input or a missing file.

Find and fix the bugs

Each box has a bug. Read the error, find the cause, fix it, and press **Run**.

Your turn: fix the bug

```
marks = [70, 85, 90]
for i in range(1, len(marks) + 1):
    print(marks[i])
```

Your turn: fix the bug

```
age = "20"
print("Next year you will be", age + 1)
```

Your turn: fix the bug

```
student = {"name": "Aman", "city": "Delhi"}
print(student["age"])
```

Your turn: make this safe with try and except

```
text = "twenty"
number = int(text)
print(number * 2)
```

Common mistakes

- **A bare** `except:` that hides every bug.
- **A huge** `try` **block.** Wrap only the line that can fail.
- `except: pass`, which throws the error away without a word.
- **Catching an error and carrying on with a wrong value.** Return `None` or stop clearly.
- **Using** `try` **where an** `if` **is clearer.** If you can simply check `if b != 0`, that is fine.
- **Expecting** `try` **to catch a** `SyntaxError`**.** It cannot.
- **Forgetting the order of** `except` **blocks.** Put the specific one before the general one.

Work out the output first, then press Run to check

```
try:
    int("x")
except Exception:
    print("general")
except ValueError:
    print("specific")
```

The general `Exception` matches first, so `specific` can never run. Put `ValueError` **above** `Exception`.

Practice

Your turn: write a function safe_divide(a, b) that returns None when b is 0

```
def safe_divide(a, b):
    pass

print(safe_divide(10, 2))
print(safe_divide(10, 0))
```

Your turn: add up the numbers and skip the items that are not numbers

```
items = ["10", "x", "25", "", "5"]
total = 0
```

Your turn: raise a ValueError if the password is shorter than 6 characters

```
def check_password(p):
    pass

try:
    check_password("abc")
except ValueError as e:
    print("Rejected:", e)
```

Homework and wrap-up

- Write a **calculator** that asks for two numbers and an operator, and handles bad numbers and division by zero.
- Write a function that reads a file and returns its lines, and prints a **friendly message** if the file is missing.
- Make a **marks reader** that reads numbers from a CSV and skips bad rows, then prints how many were bad.
- Write `get_positive_int(prompt)` that keeps asking until the user types a **positive whole number**.
- Make a **bank account** with a custom `InsufficientFunds` exception.
- Take one of your older programs and add `print` **debugging** to find a bug on purpose.
- Revise: `try`, `except`, `else`, `finally`, `raise`, common error names, how to read a traceback.

Next class

Object-oriented programming: classes and objects.